Advanced R for Econometricians (Summer 2026)

Functional R Programming — Notes and Examples

Martin Arnold, Martin Schmelzer

We need `tidyverse`.

```
library(tidyverse)
```

Anonymous Functions: Native Lambda `\(x)` vs. Legacy `~`

Since R 4.1 the **native lambda** `\(x)` is the preferred shorthand for anonymous functions:

- `\(x) expr` is equivalent to `function(x) expr`
- Use `\(x)` in new code; the twiddle (`~`) shorthand with `.x` is still valid but considered legacy

```
map_dbl(mtcars, \(x) length(unique(x)))   # preferred
map_dbl(mtcars, ~ length(unique(.x)))       # legacy, still works
```

Good rule of thumb: if a function spans multiple lines or uses `{...}`, give it a name.

`purrr::map()`

- Note that `map()` always returns a list.
- There is a ton of variations of `map()`, see `?map`. We can't cover everything. But how cool is this one:

```
# another example of the map_*() family
map_if(ggplot2::economics, is.numeric, mean)
```

`purrr::map_dbl()` and `purrr::map_int()`

Of course, the '`*`' in `map_*()` must match the return type of the functions used for mapping!

- Note that `map_vec()` (purrr >= 1.0.0) handles any vector type including dates, factors, and S3 vectors.

`purrr::map_*()` — Element Extraction and `pluck()`

Use `map_*()` with a string or integer to extract named or positional elements from a list:

```
x <- list(
  list(-1, x = 1, y = c(2), z = "a"),
  list(-2, x = 4, y = c(5, 6), z = "b"),
  list(-3, x = 8, y = c(9, 10, 11))
)

map_dbl(x, "x")   # by name
map_dbl(x, 1)     # by position
```

For deep extraction from a **single** element use `pluck()`:

```r
pluck(x, 2, "y")                    # returns c(5, 6)
pluck(x, 3, "z", .default = NA)     # safe: returns NA if missing
```

`pluck()` is preferable to `[[` chains for nested structures: it is safe (no error when `.default` is supplied) and readable.

`purrr::map_dbl()` — Producing Atomic Vectors (Task)

Solution to the task:

Please do not use a `for` loop!

As discussed in *Advanced R concepts*, everything we do in `R` involves function calls. `[[` is a primitive function—a fast C implementation for `list` subsetting. We can thus use it to iterate over `x` and subset by position/name.

```r
# 1.
sapply(x, "[[", "x")
# 2.
sapply(x, "[[", 1)
```

`purrr::map_*()` — Producing Atomic Vectors (`.default`)

Note that `.default = NA` requires your subsequent code to be compatible with `NA` values.

`purrr::map_*()` — Exercises

1. Note that
   - `map(1:3, ~ runif(2))` evaluates `runif()` with argument `n = 2` in *every* iteration since `~` converts the formula to an anonymous function (`function(x) runif(2)`).
   - `map(1:3, runif(2))` evaluates `runif(2)` *only once* and cannot do the mapping as `runif(2)` cannot be transformed to a function, see `?purrr::as_mapper()`. Thus `NULL` is returned in every iteration.

2. ```r
   library(ggplot2)

   trials_df <- tibble(p_value = map_dbl(trials, "p.value"))

   trials_df |>
     ggplot(aes(x = p_value, fill = p_value < 0.05)) +
     geom_histogram(binwidth = .025) +
     ggtitle("Distribution of p-values for random Poisson data.")
   ```

3. ```r
   models <- map(formulas, lm, data = mtcars)
   ```

Case Study — Model Fitting with `purrr`

Read in the data and split by `Drive`.

```r
# (make sure to specify the correct path below)
cars2018 <- readr::read_csv("../data/cars2018.csv")
by_drive <- split(cars2018, cars2018$Drive)
```

- **`purrr`-style approach** (using native `|>` and `\(data)` lambda):

```r
by_drive |>
  map(\(data) lm(MPG ~ Cylinders, data = data)) |>
  map(coef) |>
  map_dbl(2)
```

- **`apply()`-style:**

```r
models <- lapply(by_drive, function(data) lm(MPG ~ Cylinders, data = data))
vapply(models, function(x) coef(x)[[2]], double(1))
```

- **`for()` loop:**

```r
slopes <- double(length(by_drive))
for (i in seq_along(by_drive)) {
  model <- lm(MPG ~ Cylinders, data = by_drive[[i]])
  slopes[[i]] <- coef(model)[[2]]
}
slopes
```

Additional notes:

- `purrr` code is most accessible since each line encapsulates a single step and `map_*` conveys what is done in each step.
- Moving from `purrr` to base R we see that the number functions which iterate decreases while each iteration becomes increasingly complicated:
  - Using `purrr` we iterate 3 times (`map()`, `map()` and `map_dbl()`)
  - The `apply()` approach iterates twice (`lapply()` and `vapply()`)
  - Everything may be done in a single (but messy!) `for()` loop

Predicate Functionals: `keep()`, `discard()`, and friends

These functions filter list elements based on a **predicate** (a function returning `TRUE`/`FALSE`):

- `keep(.x, .p)` — keep elements where `.p` returns `TRUE`
- `discard(.x, .p)` — drop elements where `.p` returns `TRUE`
- `compact(.x)` — drop `NULL` elements (shorthand for `discard(.x, is.null)`)
- `every(.x, .p)` — `TRUE` if `.p` is `TRUE` for all elements
- `some(.x, .p)` — `TRUE` if `.p` is `TRUE` for at least one element

```r
models <- map(split(mtcars, mtcars$cyl), \(d) lm(mpg ~ wt, data = d))
keep(models, \(m) summary(m)$r.squared > 0.75)
compact(list(1, NULL, 3, NULL, 5))
every(models, \(m) nobs(m) >= 3)
```

Combining Map Output: `list_rbind()` and `list_c()`

`map_dfr()` and `map_dfc()` are **superseded** in purrr >= 1.0.0. Use explicit combination instead:

```r
# Superseded (avoid in new code):
map_dfr(split(mtcars, mtcars$cyl), \(d) broom::tidy(lm(mpg ~ wt, d)))

# Modern replacement:
split(mtcars, mtcars$cyl) |>
  map(\(d) broom::tidy(lm(mpg ~ wt, d))) |>
  list_rbind()
```

```r
# list_c() collapses a list of atomic vectors into one:
map(1:3, \(n) runif(n)) |> list_c()
```

Adverbs: `safely()` and `possibly()`

Adverbs modify functions so they never throw errors — essential for robust iteration.

- `safely(f)` wraps `f` and returns `list(result, error)` — always succeeds
- `possibly(f, otherwise)` returns `otherwise` on error — simpler API

```r
safe_log <- safely(log)
results <- map(list(10, -1, "a"), safe_log)
results |> map("result") |> compact()   # extract successes
results |> map("error")  |> compact()   # extract errors

# possibly() is simpler when you just need a fallback:
safe_log2 <- possibly(log, NA_real_)
map_dbl(list(10, -1, "a"), safe_log2)
```

	`safely()`	`possibly()`
On error	`list(result = NULL, error = <cnd>)`	returns `otherwise`
Use when	need to inspect errors	just need a fallback

`purrr::walk()` *(excluded from slides — supplementary)*

Assignment to an environment is a commonly used side-effect:

```r
# ABC(1) => A <- 1, ABC(2) => B <- 2, ...
ABC <- function(x) {
  assign(LETTERS[x], x, envir = globalenv())
}

# both return invisibly:
invisible(lapply(1:3, ABC))
walk(1:3, ABC)

# `walk()` in functional-style 'workflow' (un-silenced :) )
1:26 |> walk(ABC) |> cat()
```

`purrr::walk2()` *(excluded from slides — supplementary)*

Writing to disc is another side-effect. We need to map over two arguments: an object and a path.

```r
cars2018 <- readr::read_csv("../data/cars2018.csv")

t <- tempfile()     # temporary path
dir.create(t)       # create folder at t

# list of splits
tm <- split(cars2018, cars2018$Transmission)

# generate paths
paths <- file.path(t, paste0(names(tm), ".csv"))
```

```r
# walk over two arguments.
walk2(tm, paths, write.csv)

# inspect temporary folder
dir(t)
```

purrr::imap() *(excluded from slides — supplementary)*

```r
cars2018 |>
  select_if(is.numeric) |>
  imap_chr(\(x, y) paste0("The Mean of ", y, " is ", mean(x)))
```

purrr::pmap() — Named and Data Frame Inputs

Use **named lists** or **data frames** so `pmap()` matches arguments by name — especially useful with many or complex inputs:

```r
# Named list: pmap() matches 'trim' by name, 'x' is passed additionally
trims <- c(0, 0.1, 0.2, 0.5)
x <- rcauchy(1000)
pmap_dbl(list(trim = trims, na.rm = FALSE), .f = mean, x = x)

# Data frame: column names match argument names automatically
params <- tibble::tribble(
  ~ n, ~ min, ~ max,
   1L,     0,     1,
   2L,    10,   100
)
pmap(params, runif)
```

purrr::pmap() — Exercises

1. `modify()` is a shortcut for `x[[i]] <- f(x[[i]]); return(x)`. So every row is filled with its *first* value.
2. This a a good example of a quite complex operation which is relatively easy to comprehend by only looking at the code.

   ```r
   nm <- names(trans)
   cars2018[nm] <- map2(trans, cars2018[nm], \(f, var) f(var))
   ```

   - The functions in `trans` are intended to modify certain columns in `cars2018` (column names are provided as entry names in `trans`)
   - `map2()` runs over a named `list` of functions, `trans`, and a set of columns in `cars2018` which is obtained by subsetting using the function `names`
   - An anonymous function is used to call transform columns using the corresponding functions in `trans`
   - The results are used to replaced the original columns.
3.
   - Note that both approaches yield the same result
   - `map()` iterates over the variable names and calls the corresponding functions. Usage of `[[` and `.x` in the formula interface is pretty compact and conveys that columns of `cars2018` are modified.

- Using the iteration over functions and variables in `map2()` allows us to use expressive variable names (`f`, `var`) which is not possible for the `map()` approach which iterates over names.
- We could've also used the formula interface in `map2()` (which is even more compact) but the result looks rather cryptic:

```
cars2018[nm] <- map2(nm, cars2018[nm], ~ .x(.y))
```

- You should decide what you consider the most comprehensible!

Function Factories — Lazy Evaluation and `force()`

R uses **lazy evaluation**: function arguments are not evaluated until first used. This can cause unexpected bugs in function factories:

```
nth_root <- function(n) function(x) x^(1/n)
n <- 2; sq <- nth_root(n); n <- 16
sq(64)   # Bug! n is 16 when sq() is called: 64^(1/16) = 1.30
```

Fix: call `force(n)` inside the factory to evaluate `n` immediately when the factory runs:

```
nth_root <- function(n) {
  force(n)
  function(x) x^(1/n)
}
n <- 2; sq <- nth_root(n); n <- 16
sq(64)   # Correct: 8
```

Rule of thumb: always `force()` factory arguments that the manufactured function depends on.

Note: `force(x)` is simply `x` — it exists to make the intent explicit.

Function Factories — Stateful Functions

Function factories allow us to create **functions with a memory** (state). The state is tracked in the execution environment of the factory, which is **separate** for each manufactured function:

```
new_counter <- function() {
  i <- 0
  function() {
    i <<- i + 1   # <<- modifies i in the enclosing (factory) environment
    i
  }
}

counter_one <- new_counter()
counter_two <- new_counter()

replicate(2, counter_one())   # 1, 2
replicate(5, counter_two())   # 1, 2, 3, 4, 5
```

The `<<-` operator modifies a variable in the **enclosing environment** rather than creating a new local binding. Inspect with `rlang::env_print(counter_one)` to see the separate environments.

> **Caution:** Use stateful functions with moderation. For managing multiple variables, the R6 system is more appropriate.